Case Study 01: Examination Answer Evaluation

1. Problem Statement

Develop a Python program to automate the evaluation of an objective-type multiple-choice examination. The program compares a student's submitted answers against a predefined answer key of fixed length, handles unanswered questions denoted by a specific marker (-), applies positive marking (+2 marks) for correct responses, applies negative marking (-0.5 marks) for incorrect responses, ensures the total score does not fall below zero, and displays an itemized evaluation summary.

2. Objectives

- To validate that the number of submitted responses matches the length of the predefined answer key.
- To iterate concurrently across submitted answers and the answer key using Python's built-in zip() function.
- To implement conditional scoring logic accounting for correct responses, negative marking, unattempted questions, and a zero-score floor constraint.
- To output an organized summary showing correct counts, incorrect counts, unattempted counts, and total marks secured.

3. Assumptions

- The examination has a fixed answer key containing 5 questions: ['A', 'C', 'B', 'D', 'A'].
- Each question carries 2 marks for a correct answer, -0.5 marks for an incorrect answer, and 0 marks for an unattempted question (-).
- Inputs are case-insensitive and entered on a single line separated by whitespace.
- The minimum possible final score is bounded at 0 (max(0, score)).

4. Algorithm

- **Step 1:** Start the program.
- **Step 2:** Define the reference answer key list: key = ["A", "C", "B", "D", "A"].

- **Step 3:** Prompt the user to input responses separated by spaces, convert them to uppercase, and split them into a list named answers.

- **Step 4:** Check if len(answers) is not equal to len(key). If true, print "Please enter exactly 5 answers" and terminate; otherwise, proceed.

- **Step 5:** Initialize counters: correct = 0, wrong = 0, blank = 0, and score = 0.

- **Step 6:** For each pair (answer, correct_answer) in zip(answers, key):

    - If answer == "-": increment blank by 1.

    - Else if answer == correct_answer: increment correct by 1 and increment score by 2.

    - Else: increment wrong by 1 and decrement score by 0.5.

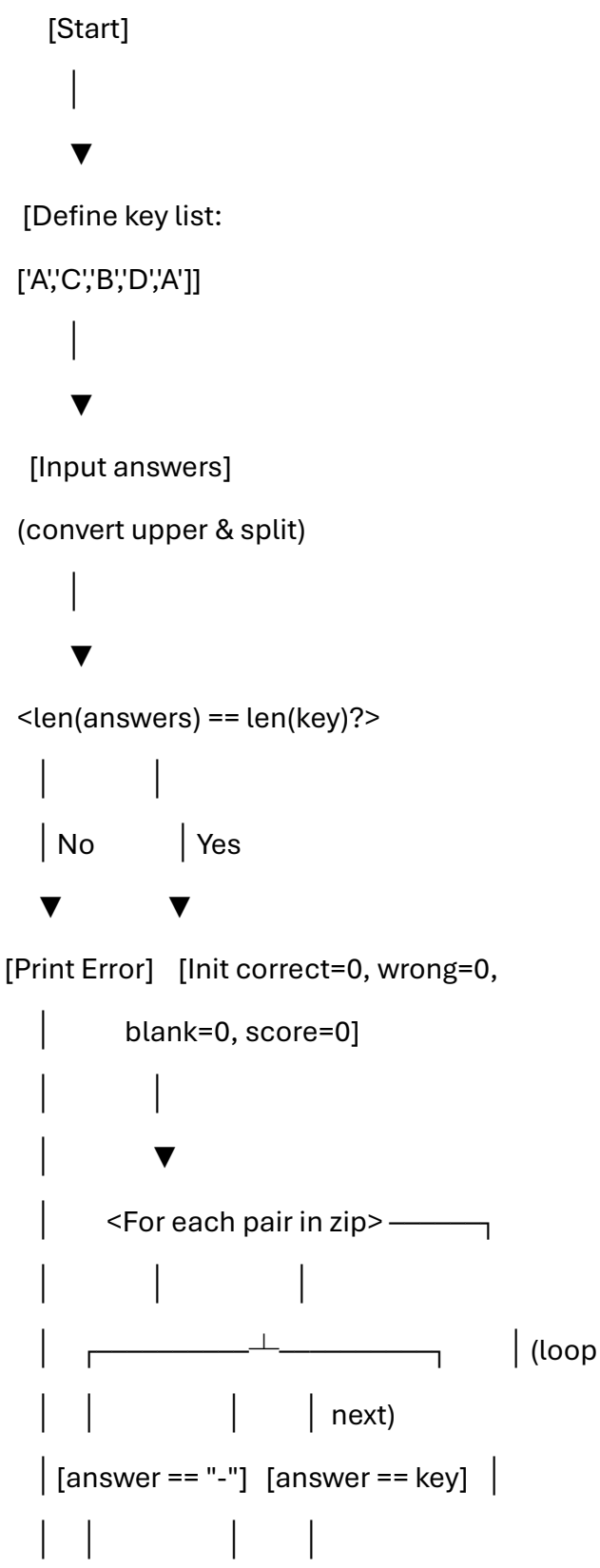
- **Step 7:** Set score = max(0, score) to prevent negative cumulative marks.

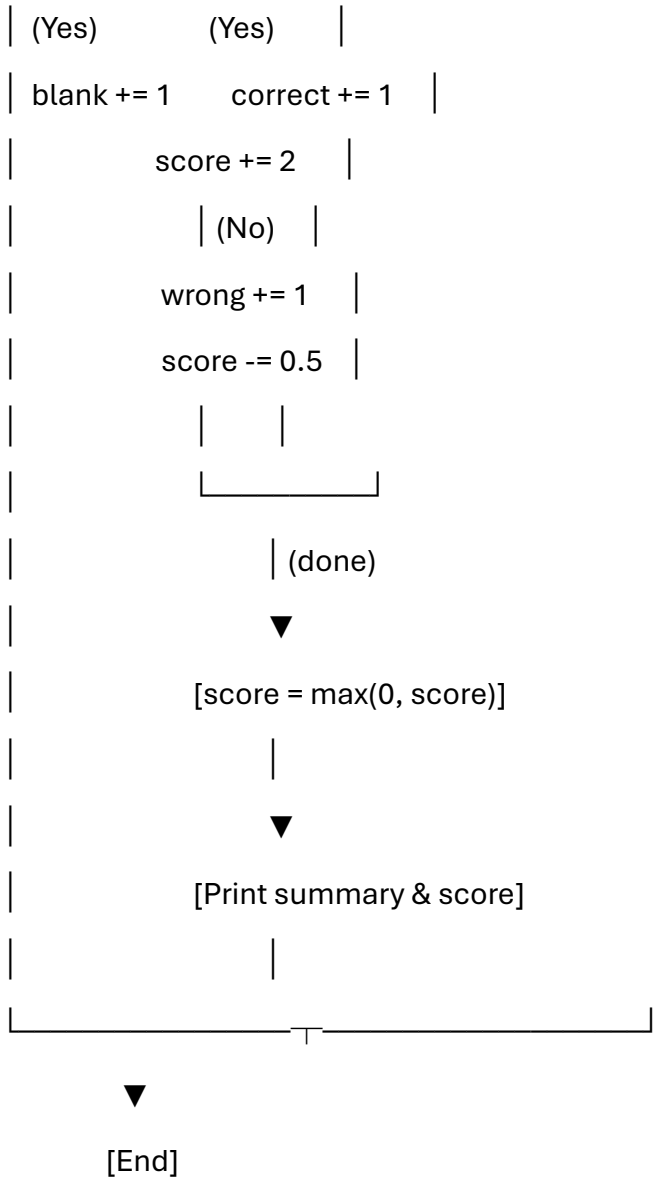
- **Step 8:** Display counts of correct, wrong, blank, and the score out of the maximum possible marks (len(key) * 2).

- **Step 9:** Stop the program.

5. Flowchart Logic / Diagram

Plaintext


[Start]
[Define key list:
['A','C','B','D','A']]
[Input answers]
(convert upper & split)
<len(answers) == len(key)?>
No
Yes
[Print Error]
[Init correct=0, wrong=0,
blank=0, score=0]
<For each pair in zip>
[answer == "-"]
[answer == key]
(loop
next)



6. Pseudocode

Plaintext

```
BEGIN
  SET key = ["A", "C", "B", "D", "A"]
  PRINT "Enter 5 answers (use - for blank), separated by spaces:"
```

```
READ input_string
SET answers = SPLIT(TO_UPPERCASE(input_string))

IF LENGTH(answers) != LENGTH(key) THEN
  PRINT "Please enter exactly 5 answers"
ELSE
  SET correct = 0
  SET wrong = 0
  SET blank = 0
  SET score = 0.0

  FOR EACH (answer, correct_answer) IN ZIP(answers, key) DO
    IF answer == "-" THEN
      blank = blank + 1
    ELSE IF answer == correct_answer THEN
      correct = correct + 1
      score = score + 2.0
    ELSE
      wrong = wrong + 1
      score = score - 0.5
    END IF
  END FOR

  score = MAXIMUM(0.0, score)

  PRINT "Correct:", correct
  PRINT "Wrong:", wrong
  PRINT "Unanswered:", blank
```

```
    PRINT "Final score:", score, "out of", LENGTH(key) * 2
  END IF
END
```

7. Python Source Code

Python

```
# Examination Answer Evaluation System
# Define master answer key
key = ["A", "C", "B", "D", "A"]

# Read and sanitize candidate answers
answers = input("Enter 5 answers (use - for blank), separated by spaces:
").upper().split()

# Input validation for length match
if len(answers) != len(key):
  print("Please enter exactly 5 answers")
else:
  correct = wrong = blank = 0
  score = 0.0

  # Evaluate responses against key
  for answer, correct_answer in zip(answers, key):
    if answer == "-":
```

```
        blank += 1
    elif answer == correct_answer:
        correct += 1
        score += 2
    else:
        wrong += 1
        score -= 0.5

# Floor score at 0
score = max(0.0, score)

# Display evaluation summary
print("Correct:", correct)
print("Wrong:", wrong)
print("Unanswered:", blank)
print("Final score:", score, "out of", len(key) * 2)
```

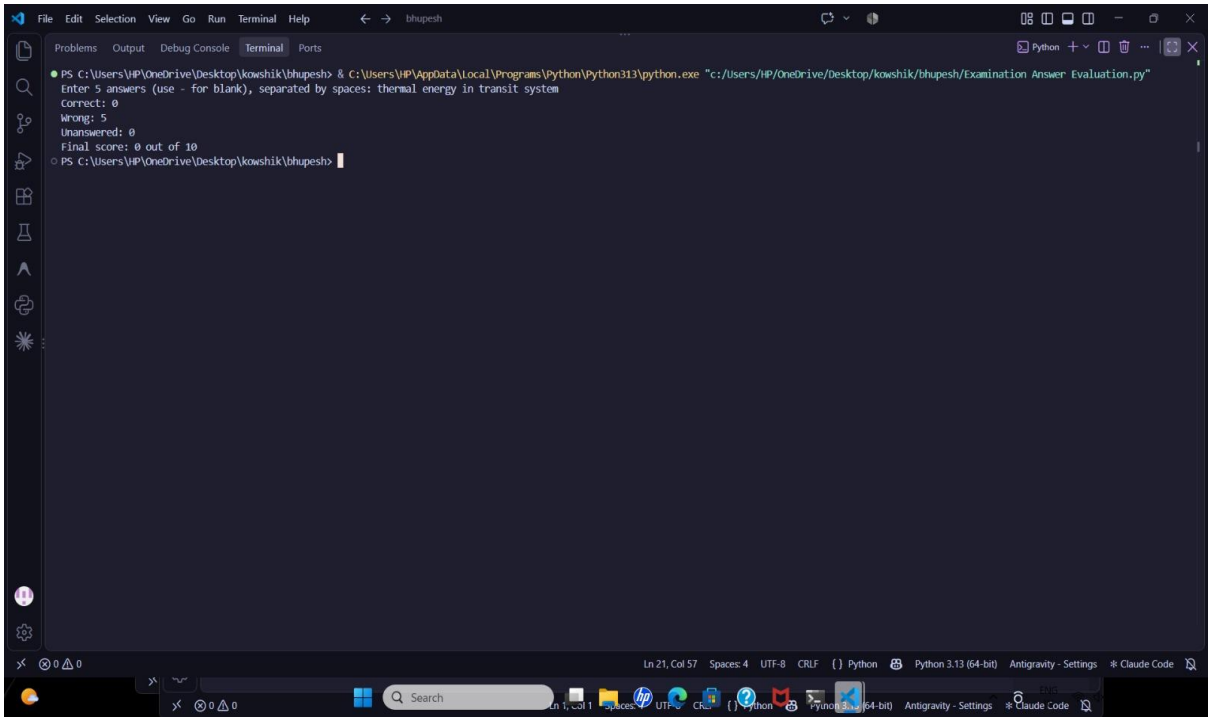
8. Test Cases

Test Case ID	Input Given	Expected Output	Actual Output	Status
TC_01 (All Correct)	A C B D A	Correct: 5	Correct: 5	Pass

Test Case ID	Input Given	Expected Output	Actual Output	Status
		Wrong: 0 Unanswered: 0 Final score: 10 out of 10	Wrong: 0 Unanswered: 0 Final score: 10.0 out of 10	
TC_02 (Mixed Responses)	A C - B A	Correct: 3 Wrong: 1 Unanswered: 1 Final score: 5.5 out of 10	Correct: 3 Wrong: 1 Unanswered: 1 Final score: 5.5 out of 10	Pass
TC_03 (Negative Score Floor)	B A A A C	Correct: 0 Wrong: 5 Unanswered: 0	Correct: 0 Wrong: 5 Unanswered: 0	Pass

Test Case ID	Input Given	Expected Output	Actual Output	Status
		Final score: 0 out of 10	Final score: 0.0 out of 10	
TC_04 (All Unanswered)	- - - - -	Correct: 0 Wrong: 0 Unanswered: 5 Final score: 0 out of 10	Correct: 0 Wrong: 0 Unanswered: 5 Final score: 0.0 out of 10	Pass
TC_05 (Invalid Count)	A B C	Please enter exactly 5 answers	Please enter exactly 5 answers	Pass

9. Output Screenshots



10. Conclusion

The Examination Answer Evaluation program was implemented and verified successfully. It applies list operations, string manipulation (upper(), split()), sequential iteration using zip(), and conditional branching to grade responses with negative marking rules and input validation.